

NodeJS 中 CommonJS 和 ESM module 混用时的各种坑解决方案



MoyuScript
独立软件开发者

关注他

29 人赞同了该文章 >

前言

NodeJS 中，目前有两种标准的模块引入模式，一种是旧的 CommonJS (CJS)，另外一种是现代的 ESM module (ESM)。

有的时候，我们不得不混用这两种引入模式（一些第三方库仅支持 ESM），这时候就会产生一些坑，比如如果尝试 require (CJS) 一个 ESM 文件时，就会报错。

这篇文章主要是我对这两种模块引入方式一些坑的一些总结。

CJS 和 ESM 单用

CJS

CJS 文件定义方法：

1. 默认 .js 后缀的文件为 CJS。
2. .cjs 后缀的文件强制为 CJS。
3. package.json 中 type="commonjs" 时，.js 文件会被认为是 CJS。

模块定义：

```
// ./lib.js
module.exports = function() {
  console.log('hello');
}
```

```
// ./lib2.js
exports.world = function () {
  console.log("world");
};
```

模块引入：

```
const lib = require("./lib.js");
const { world } = require("./lib2.js");

lib(); // "hello"

world(); // "world"
```

CommonJS 示例

codesandbox.io/s/commonjs-shi-li-4nebv?file=/index.js

CommonJS 参考

nodejs.org/dist/latest-v16.x/docs/api/modules.html

```
const exports = module.exports;
```

因此，如果混用两者，容易产生一些问题，就比如下面这段代码：

```
// ./lib.js
module.exports = function () {
  console.log('hello');
}

// module.exports 的引用已被改变，因此这里的 exports 仍然指向改变前的 module.exports，
exports.world = function () {
  console.log('world');
}
```

如果尝试引入该模块，会出现以下结果：

```
const module = require('./lib.js');

module(); // "hello"
module.world(); // 出错
```

因此，实际使用中一般建议最好不要同时使用 `module.exports` 和 `exports`，如果一定要同时使用，可以在重新赋值 `module.exports` 后，更新 `exports` 对应引用：

```
module.exports = function () {
  console.log('hello');
}

exports = module.exports;

// 这样就没问题了
exports.world = function () {
  console.log('world');
}
```

ESM

ESM 文件定义方法：

1. package.json 中 type="module" 时，.js 后缀文件会被识别为 ESM。
2. .mjs 后缀的文件会被强制识别为 ESM。

模块定义：

```
// ./lib.js

export default function () {
  console.log("hello");
}

export function world() {
  console.log("world");
}
```

模块引入：

```
import hello from "./lib.js";
import { world } from "./lib.js";
```

```
world(); // "world"
```

```
// 动态引入
```

```
import("./lib.js").then((module) => {  
  console.log(module); // { default: Function, world: Function }  
});
```

ESModule 示例

codesandbox.io/s/esmodule-shi-li-4foj5i?file=/lib.js

ESModule 参考

nodejs.org/dist/latest-v16.x/docs/api/esm.html

坑1: __filename, __dirname 无法使用

这两个变量仅能在 CJS 中使用，参考：[No __filename or __dirname \(nodejs.org\)](#)。

ESM 中等效代码：[dirname-filename-esm/index.js at master · rhyds/dirname-filename-esm \(github.com\)](#)

CJS 和 ESM 混用

CJS 引入 ESM

可以使用动态 import 来引入。

例子：

```
// lib.mjs  
export function hello() {  
  return 'world';  
}  
  
// main.js  
import('./lib.mjs').then((lib) => console.log(lib.hello())); // 输出: world
```

坑1: ESM 编译注意

如果你写了个库，使用静态 import 导入了 ESM 的第三方库（比如 [got](#)），那么你的代码在编译时，不能编译为 CJS（比如 tsconfig.json 中的 \$.compilerOptions.module 必须为 "ES6"，而不是 "CommonJS"），否则编译后的代码无法使用。

坑2: 配置文件报错

通常根目录下会存放一些第三方库的配置文件（比如 .eslintrc），如果你是 package.json 中 type="module"，那么会默认项目下所有 JS 文件为 ESM，然而一些第三方库引入配置文件的方式是 CJS，这就会导致报错。

这里有两种解决方式：

一、package.json 中不配置 type，项目中 ESM 文件全部使用 .mjs 结尾（不推荐）

将所有 ESM 文件改为 .mjs 结尾，其他普通 .js 文件就会被默认识别为 CJS，这种方法确实能解决问题，但是改成 .mjs 会比较麻烦，改动较大，而且如果你使用了 TS 进行编译，它编译出来的文件还是 .js 结尾（貌似没法修改），同样会导致报错，因此这种方式不推荐。

二、配置文件使用 .cjs 结尾（强烈推荐）

ESM 引入 CJS

```
// ./lib.js
exports.world = function () {
  console.log("world");
};

// ./index.mjs
import lib from './lib.js';

console.log(lib); // { world: Function }

lib.world(); // "world"
```

ESM 引入 CJS 示例

codesandbox.io/s/esm-yin-ru-cjs-bs5yub?file=/index.mjs

ESM 引入 CJS 没什么太大的坑，module.exports 的值是什么，import 的时候就是什么，可以自己尝试一下。

结语

ESM 取代 CJS 是必然的事了，现在还是过渡阶段，因此难免会产生一些兼容问题，以后开发的新的第三方库都尽量要使用 ESM 了。

编辑于 2022-06-14 20:18

[Node.js](#) [CommonJS](#) [JavaScript](#)

▲ 赞同 29 ▼ ● 7 条评论 ↗ 分享 ❤ 喜欢 ★ 收藏 📄 申请转载 ...



理性发言，友善互动

7 条评论

默认 最新



海胆

可惜electron一直坚持cjs，太恶心了🤔

2023-01-17

🗨 回复 ❤ 6



Catsup

electron 除了 preload.js 需要是 cjs，其他都可以用 mjs，就是 __dirname 烦人一点

2023-12-30

🗨 回复 ❤ 1



鱼钩的前端

commonjs nodejs 上用的多啊，esmodule 才出来多久都不普及。

2023-08-18

🗨 回复 ❤ 喜欢



Unreal

ESM 引入 CJS 怎么会没坑呢，坑多到你怀疑人生好吗

2023-08-21

🗨 回复 ❤ 3



鱼与熊掌

为什么没用ts

2022-04-08

🗨 回复 ❤ 1



鱼钩的前端

ts 和 js 混用也很糟心，一定要有 .d.ts

...



郭云水

对于作者最后讲的ESM引入CJS，我想补充的时要把导入的文件名后缀改为.cjs，要不然会报错：import lib from './lib.js';
^^^

SyntaxError: The requested module './lib.js' does not provide an export named 'default'
把上边的lib.js后缀名改为lib.cjs则不会爆以上的错误，我的运行环境是
node --version v20.12.2

欢迎大家一块讨论，共同进步

2024-09-23

回复 1

推荐阅读

绝对是讲的最清楚的-NodeJS
模块系统

NodeJS目前有两个系统：一套是CommonJS(简称CJS)，另一套是ECMAScript modules(简称ESM);
本篇内容主要三个话题： 1. CommonJS的内部原理 2. NodeJS平台的ESM模块系统 3....

chenb...

发表于前端好好学



2020年我们可以在Node中使用
ES Modules了吗

转转技术团...

发表于大转转FE



浅析当下的 Node.js
CommonJS 模块系统

Awee

发表于前端深夜告...



【译】Node 模块之战：
CommonJS 和 ES Mod

泡菜